INTENT ENFORCEMENT FOR AI AGENTS

FlowGuard

How to use it, the role ArmorIQ plays, and how it differs from typical industry approaches

Part	What you will find
1. The idea	Why agents need intent checks, and the one example that explains it.
2. How to use it	Prerequisites, the quickstart, commands, reading decisions, writing good plans.
3. The role of ArmorIQ	What ArmorIQ does, what the gateway adds, and what you depend on.
4. How it differs	Side-by-side with guardrails, access control, approvals, logging, and model safety.
5. Rollout checklist	What to confirm before you publish or go live.

Scope and assumptions

FlowGuard is described here as an intent-enforcement gateway that sits in front of MCP servers and uses ArmorIQ to sign and check each agent's plan. Command names, ports and behaviours come from the public **IntentGate** reference project (MIT licence, by Manjunath Irukulla, built for the ArmorIQ x ElevenLabs Flow State hackathon). If FlowGuard ships under a different package name or behaves differently, update Section 2 before you share this. ArmorIQ details come from its public website, documentation and GitHub. This guide has not been tested against a live deployment.

PART 1

The idea

An AI agent acts on text it reads. A README, a web page, a support ticket or a caller on the phone can all carry hidden instructions, and attackers write some of that text. If the agent can read files and also send data out, a single poisoned line can turn a harmless task into a leak.

Intent enforcement adds a check between the agent's reasoning and the action:

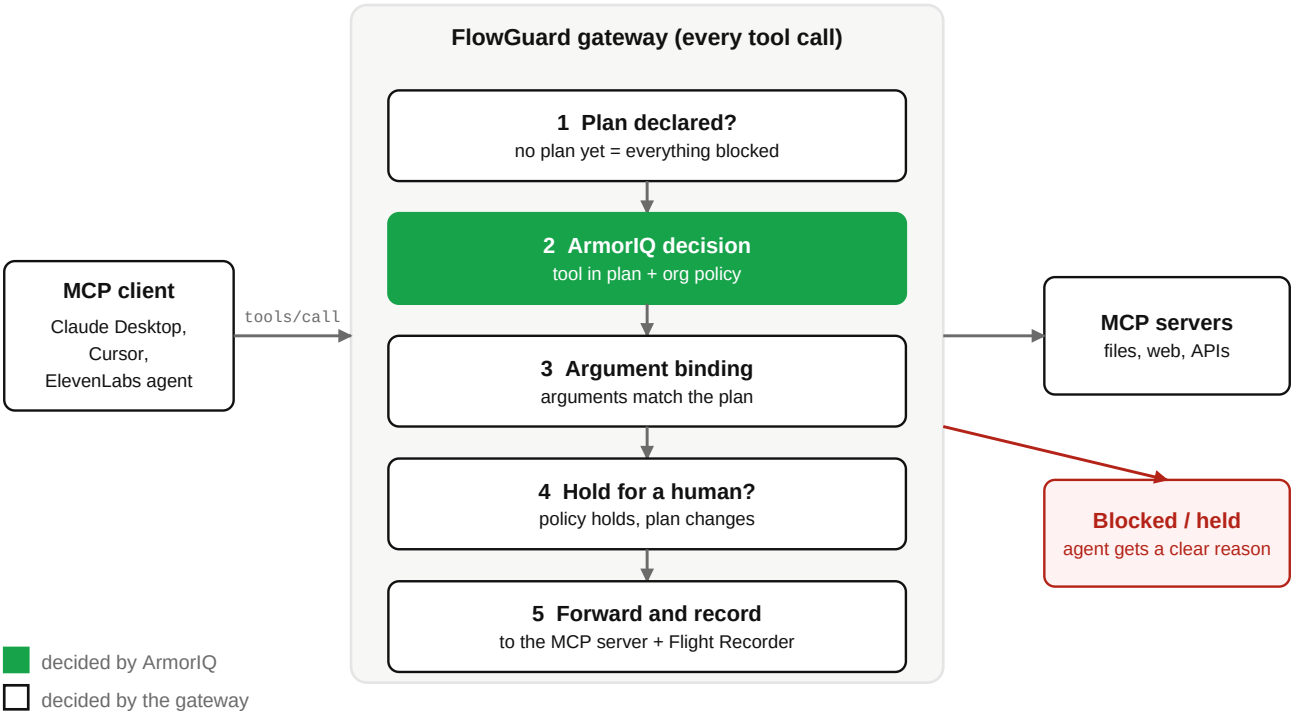
- **Declare.** Before acting, the agent states its plan: which tools it will use and with what arguments.
- **Sign.** ArmorIQ signs that plan, so it becomes the agreed boundary for the run.
- **Check.** Every tool call is compared with the plan. Anything outside it is blocked, or held for a human to approve.

One session, three decisions

Agent tries to...	Result	Why	Decided by
Read README.md (in plan)	**Allowed**	Tool and file are in the plan, and org policy allows it.	ArmorIQ backend
Read .ssh/id_rsa (same tool, different file)	**Blocked**	The path differs from what the plan declared.	Gateway argument binding
Send an HTTP POST to an attacker	**Blocked**	The tool is not in the plan.	ArmorIQ backend

The row in the middle matters most. A plan that only names a tool lets that tool run on anything. Binding the arguments makes the plan mean what it says. Section 3 explains which part of the system provides that.

How a call travels



PART 2

How to use it

Before you start

- **Node.js 20 or newer.**
- **An ArmorIQ account and API key** from platform.armoriq.ai. Without a key you can only try things in an unsigned offline mode.
- **A client to protect:** Claude Desktop, Cursor, or an ElevenLabs voice agent.
- *Optional:* an Anthropic API key (for the built-in demo agent) and an ElevenLabs API key (for spoken alerts).

Quickstart

Step	What to do
1	Create your ArmorIQ account and copy the API key into the environment variable `ARMORIQ_API_KEY`. Never commit it.
2	On the ArmorIQ platform, register your agent and the MCP servers it uses, then create a policy: allow the read-only tools you need, block risky ones (writing files, posting to the web).
3	Protect your client. For Claude Desktop run `npx intentgate-mcp init` (add `--client cursor` for Cursor). It wraps the MCP servers already in your config.
4	Run `npx intentgate-mcp doctor`. It checks keys, servers and ports, and asks ArmorIQ what it will do for each tool.
5	Open the Flight Recorder at `http://localhost:4747` to watch plans, allows, blocks and holds as they happen.
6	For voice agents, run `npx intentgate-mcp serve --token <secret>`, host it over HTTPS, and add it to the ElevenLabs agent as an MCP server with that token.
7	To undo everything, run `npx intentgate-mcp uninstall`. It restores your original config.

If FlowGuard publishes its own package name, replace `intentgate-mcp` in every command above.

Command reference

Command	What it does
`npx intentgate-mcp init`	Wraps the MCP servers in Claude Desktop (or Cursor with --client cursor).
`npx intentgate-mcp agent`	Runs a demo where a real Claude agent meets a poisoned workspace, with enforcement on.
`... agent --passthrough`	Same demo with no enforcement, so you can compare honestly.
`... playground`	Hosts the demo page and the Flight Recorder (what a live demo site runs).
`... serve --token <secret>`	Streamable HTTP endpoint for ElevenLabs agents.
`... doctor`	Checks keys, servers, ports and the decision ArmorIQ will make per tool.
`... uninstall`	Restores your original client config.
`--offline (flag)`	Try everything unsigned, without an ArmorIQ key. For trials only, not protection.

Reading a decision

Outcome	Meaning	What you do
Allowed	The tool and arguments match the signed plan and org policy.	Nothing.
Blocked	No plan declared yet, the tool is not in the plan, or the arguments differ from the plan.	Check the reason in the Flight Recorder. If the call was legitimate, the plan was too narrow.
Held for a human	A policy hold, or the agent asked to change its plan.	Approve or deny in the Flight Recorder. On voice calls the agent gets an answer within five seconds so the call does not time out, and your decision is recorded when it lands.

Writing good plans

- Declare only the tools the task needs. Anything extra is attack surface.
- Use exact paths or narrow patterns. A plan that allows a URL of * allows every URL, so it gives loose protection.
- Expect everything to be blocked until a plan exists. That is by design.
- Start with read-only tools in your policy, watch the Flight Recorder for a week, then widen carefully.

PART 3

The role of ArmorIQ

ArmorIQ is a security platform for autonomous AI agents, founded in 2025 and based in San Francisco. Its pitch is **live intent assurance**: a control layer between an agent's reasoning and its execution, so an agent can plan and adapt freely but no action runs unless it fits the approved task. Its own wording for the difference from guardrails is short: guardrails stop bad responses, intent enforcement stops bad actions.

Who does what

Responsibility	Provided by
Signs the declared plan and issues an intent token	**ArmorIQ**
Decides every tool call: is the tool in the plan, and does org policy allow it	**ArmorIQ backend** (called sdk mode)
Agent and MCP-server registry, policy builder, decision counts and trace	**ArmorIQ platform console**
Makes all of the above work for clients that cannot run SDK code (Claude Desktop, Cursor, ElevenLabs agents)	**FlowGuard gateway**
Argument binding: normalised paths, globs, arguments must match the plan	**FlowGuard gateway**
Human approval holds, spoken alerts, Flight Recorder	**FlowGuard gateway**

Why ArmorIQ is the important piece

- **It is an independent decision-maker.** The verdict does not come from the model being protected or from the app being protected. A manipulated model cannot talk its way past it.
- **The plan is signed.** Signing makes the plan tamper-evident, so the boundary the agent agreed to is the boundary that is enforced.
- **Decisions land in one place.** Because the checks run on ArmorIQ's backend, they appear in its trace by construction, which gives you an audit trail for free.
- **Policy is central.** Organisation-wide rules apply the same way across agents and clients.

What you depend on

- **Availability and latency.** In sdk mode every decision is made on ArmorIQ's server. Ask ArmorIQ about uptime commitments and what happens to agents if it is unreachable.
- **Local verification is not possible yet.** The reference project reports that the backend does not publish its verification key, so checks cannot run fully offline.
- **Commercial terms and data handling.** Confirm pricing, limits, and what tool-call data ArmorIQ stores, before you promise anything to customers.
- **Ecosystem.** ArmorIQ's GitHub organisation lists enforcement plugins for several agent platforms (OpenClaw, Gemini CLI, Claude, Copilot Studio, and an NVIDIA OpenShell middleware), which suggests the same approach can follow you if your agent stack changes.

PART 4

How it differs from typical industry approaches

Most teams protect agents with some mix of the layers below. Each is useful. The difference is *what* each one checks and *when*. Intent enforcement is meant to sit alongside them, not replace them.

Common approach	What it checks, and when	Typical weakness	How intent enforcement differs
Prompt and output guardrails, injection classifiers	The text going into or out of the model.	Probabilistic. New phrasing can slip past, and they do not control what the agent does next.	Checks the **action**, not the wording, and compares it with a fixed plan.
Identity and access control (OAuth scopes, API keys, RBAC)	Who the agent is and what it may ever do, at connection or request time.	Static. If the agent may read files and send web requests, both stay allowed for every task, whatever the user asked.	Scopes access **per task**. The same agent gets a different boundary on each run.
Human approval for every sensitive action	A person confirms each action before it runs.	Slow, and people start approving without reading when prompts are frequent.	Humans only see off-plan changes and policy holds, so each prompt is rarer and means more.
Logging and monitoring, anomaly detection	Records and alerts, usually after the action.	You learn about a leak after it happened.	Decides **before** execution. The audit trail is a by-product.
Model alignment and provider safety training	The model's own judgment inside its response.	Varies by model and version, and jailbreaks keep appearing.	The outcome does not depend on the model. In the reference tests, current Claude models refused the planted injection on their own; the gateway is for the day a model, or a scripted agent, does not.
Sandboxing and network egress rules	What the environment allows at all.	Coarse. Either blocks legitimate work or leaves gaps for allowed channels.	Finer and task-specific. Works well on top of a sandbox.

How it relates to standards

- **OWASP Top 10 for LLM Applications** lists prompt injection and excessive agency among its main risks. Limiting an agent to a declared, signed plan is a direct way to reduce excessive agency.
- **MCP authorization** (OAuth-based) answers who may connect to a server. It does not say what an agent plans to do with that connection, so the two are complementary.
- **NIST AI RMF** and similar frameworks ask for governance and traceability. An audit trail of plans and decisions supports that work, but using any tool does not by itself make you compliant. Do not make compliance claims on your site without legal review.

Honest limits

Intent enforcement is one layer, not a silver bullet

1. An injection that reaches the model *before* it declares its plan can shape that plan. Blocking all tools until a plan exists closes the usual path, not a malicious first prompt.
2. Loose plans give loose protection (for example allowing any URL).
3. ArmorIQ's plan check works at the tool level. The argument binding that closes the gap comes from the gateway, so it is only as strong as that code.
4. The reference project was built in about a week for a hackathon. Treat it as early-stage and test it before trusting it with production data.
5. Keep the other layers: least-privilege credentials, guardrails, logging and review.

PART 5

Rollout checklist

	Check	Why
[ ]	Confirm FlowGuard's real package and command names after the rename and update Section 2.	Wrong commands break the quickstart.
[ ]	If the code is derived from IntentGate, keep its MIT licence notice and credit the author.	The MIT licence requires the notice to stay.
[ ]	Get your own ArmorIQ API key and read the commercial terms and data handling.	Decisions depend on their backend.
[ ]	Register agent and MCP servers, and start with a read-only policy.	Smallest safe starting point.
[ ]	Run the demo with and without enforcement and keep both recordings.	Shows the difference honestly.
[ ]	Write tight plans: exact paths, only needed tools, no wildcard URLs.	Loose plans give loose protection.
[ ]	Host the voice gateway over HTTPS with a strong token. Keep API keys in environment variables.	The endpoint is reachable from the internet.
[ ]	Review the Flight Recorder regularly and tune plans.	Blocks reveal both attacks and plans that are too narrow.
[ ]	Make sure every website claim matches what is real (no unverified badges, uptime or compliance claims).	Public claims create legal and trust risk.

Sources

ArmorIQ: armoriq.ai and armoriq.ai/about-us
ArmorIQ platform documentation: docs.armoriq.ai/platform
ArmorIQ on GitHub: github.com/armoriq (ArmorClaw, armorGemini, armorClaude, armorCopilot, OpenShell middleware)
ArmorIQ blog, intent enforcement for OpenClaw: armoriq.substack.com
IntentGate reference project (MIT): github.com/mirukullacode/intentgate, including its docs on how it works, security model and DX notes
Company background: dealroom.co entry for ArmorIQ

This document is informational. It is not legal, security-audit or compliance advice.